Implementation and Analysis of PACD-based Attacks on RSA-CRT

Reduction to the Hidden Number Problem and Application of the LLL Algorithm

Ariașu Florian-Andrei, 342C2
Faculty of Automatic Control and Computer Science
Politehnica University of Bucharest

May 2026

Abstract

This project implements and evaluates a fault injection attack on RSA-CRT signatures, based on the paper *Revisiting PACD-based Attacks on RSA-CRT* published by Barbu, Grémy, and Lescuyer at IACR ePrint in 2024 [1]. The implementation covers two levels of attack: the classical Bellcore attack [2], which requires a single fully corrupted signature, and the attack based on the *Partial Approximate Common Divisor* (PACD) problem, which operates with partial faults by constructing a lattice and applying the LLL algorithm [4]. A simplified variant of the PACD attack was implemented, based on the Simultaneous Diophantine Approximation (SDA) approach and an implicit reduction to HNP, following Sections 3.2–3.4 of the reference article [1]. The advanced optimisations present in the article (`flatter`, BDD with Predicate) were not implemented; instead, the implementation targets RSA-512 to compensate for the lower reduction power of standard LLL compared to `flatter`. The results confirm the qualitative behaviour described in the reference article: the more known bits are available from $\tilde{s}_q$, the fewer corrupted signatures are needed to recover the secret prime factor.

Contents

1 Introduction

The RSA algorithm implementation based on the Chinese Remainder Theorem (RSA-CRT) is the gold standard in embedded and secure systems, owing to its high computational efficiency — approximately four times faster than the standard variant. However, this efficiency comes at a cost: increased vulnerability to fault injection attacks (*fault attacks*).

This project presents the implementation and evaluation of a modern attack on RSA-CRT signatures, grounded in the research published by Barbu, Grémy, and Lescuyer in 2024 [1]. The attack explores the scenario in which the injected fault is not a total one — which would allow the classical Bellcore attack [2] with a single signature — but a partial one, where only a small number of MSBs of the intermediate variable s_q are zeroed or known.

The main innovation of the reference article lies in demonstrating that the *Partial Approximate Common Divisor* (PACD) problem can be reduced to an instance of the *Hidden Number Problem* (HNP), enabling the use of modern lattice reduction algorithms (LLL [4], BKZ, `flatter`) to recover the secret prime factor. Compared to the preceding attack at CHES 2012 [3], which required 32 known bits, the authors demonstrate that 7 bits are sufficient for RSA-1024.

This implementation targets RSA-512 and uses SageMath with the standard LLL algorithm, covering Sections 3.2–3.4 of the article. The advanced optimisations (`flatter`, BDD with Predicate from Section 3.5) are not implemented, and the fault model is idealised compared to real hardware scenarios. These limitations are deliberate and compensated by reducing the RSA dimension, enabling full evaluation on consumer hardware.

2 Theoretical Background

2.1 Asymmetric-Key Cryptography and RSA

Unlike symmetric cryptography (such as AES, where a single secret key is used for both encryption and decryption), asymmetric cryptography uses a key pair: a public key (N, e), freely distributable, and a private key d, kept secret. This separation solves the key distribution problem and enables digital signatures.

In RSA [5]:

- **Key generation:** Two large primes p, q are chosen; $N = p \cdot q$, $\varphi(N) = (p-1)(q-1)$, and $d = e^{-1} \pmod{\varphi(N)}$ are computed.
- **Signing:** $s = m^d \pmod{N}$ (using the private key).
- **Verification:** $m \stackrel{?}{=} s^e \pmod{N}$ (using the public key).

The security of RSA relies on the difficulty of factoring $N = p \cdot q$. Knowing p and q, an attacker can compute $\varphi(N)$ and recover d.

2.2 RSA-CRT and Garner's Formula

Computing $m^d \pmod{N}$ is slow for N of 2048 bits. RSA-CRT accelerates the operation via the Chinese Remainder Theorem, splitting the exponentiation into two smaller computations [1]:

$$d_p = d \pmod{p-1}, \quad d_q = d \pmod{q-1}, \quad i_q = q^{-1} \pmod{p} \tag{1}$$

$$s_p = m^{d_p} \pmod{p}, \quad s_q = m^{d_q} \pmod{q} \tag{2}$$

Recombination via Garner's formula:

$$s = s_q + q \cdot [(s_p - s_q) \cdot i_q \pmod{p}] \tag{3}$$

It is important to note that in this form of Garner's formula, the component s_q is used to assemble the final signature, but the corrective term is computed modulo p. This detail determines which prime factor will be leaked: a fault injected into s_q will cause the difference between the valid and corrupted signature to be a multiple of p, enabling its recovery.

This structure is critical for security: an error in the computation of s_q propagates into s while preserving a direct mathematical relationship with the secret factor p.

2.3 The Classical Bellcore Attack

In 1997, Boneh, DeMillo, and Lipton [2] demonstrated that a single complete fault in s_q (total zeroing, $\tilde{s}_q = 0$) allows the factorisation of N. The corrupted signature becomes:

$$\tilde{s} = 0 + q \cdot [(s_p - 0) \cdot i_q \pmod{p}] \tag{4}$$

If s_q is corrupted, the faulty signature $\tilde{s}$ satisfies the relation $\tilde{s} \equiv s_p \pmod{p}$, just as the valid signature s. Therefore, the difference $s - \tilde{s}$ is divisible by p, but not by q (since $\tilde{s} \not\equiv s_q \pmod{q}$). Thus, $\gcd(s - \tilde{s}, N)$ returns the prime factor p. Conversely, a fault injected into s_p enables recovery of the factor q. In both cases, obtaining a single factor is sufficient for the complete factorisation of N.

Alternatively, if s_p is the corrupted component, the result is q. In general, $\gcd(s - \tilde{s}, N) \in \{p, q\}$, both sufficient for the factorisation of N. With a single corrupted signature and the public key (N, e), the attacker fully recovers the private key d.

2.4 The PACD Problem and Reduction to HNP

The Bellcore attack requires $\tilde{s}_q = 0$ completely. The article [1] addresses the *partial* case: if an attacker can only force L MSBs of s_q to zero, the corrupted value $\tilde{s}_q$ remains small:

$$\tilde{s}_q < \frac{q}{2^L}, \quad \rho = \text{nbits}(q) - L \quad \Rightarrow \quad \tilde{s}_q < 2^\rho \tag{5}$$

The corrupted signature becomes:

$$\tilde{s}_i = \tilde{s}_q^{(i)} + q \cdot k_i, \quad \text{where } \tilde{s}_q^{(i)} < 2^\rho \text{ is small} \tag{6}$$

Collecting t such corrupted signatures, the values $\tilde{s}_i$ are "approximate multiples" of q — this is the PACD (*Partial Approximate Common Divisor*) instance.

The novelty of the article [1] lies in reducing this PACD instance to an HNP (*Hidden Number Problem*) instance:

$$\tilde{s}_i \cdot p \pmod{N} < \frac{N}{2^L} \tag{7}$$

where p becomes the "hidden number". This reformulation enables the application of standard lattice bases used for HNP, with N on the diagonal instead of s_0 (an improvement over the generic SDA approach from [3]).

Important observation: this implementation directly constructs the matrix corresponding to this HNP reduction (with $-N$ on the diagonal), thereby implicitly performing the PACD $\rightarrow$ HNP reduction described in Section 3.4.1 of the article, even though the formal proof from Section 3.4.2 is not reproduced.

3 Implementation

3.1 Environment and Deliberate Limitations

The implementation uses **SageMath** in the CoCalc environment, chosen for its native support for large-number arithmetic and the optimised LLL implementation via `Matrix.LLL()`.

Compared to the experiments in the article [1], the implementation presents the following deliberate differences:

- **RSA dimension:** RSA-512 instead of RSA-1024 to RSA-8192. This reduces the norm of the target vector v (proportional to $p \approx 2^{256}$ rather than 2^{512}), compensating for LLL's lower reduction power compared to `flatter`.
- **Reduction algorithm:** Standard LLL (`Matrix.LLL()`) instead of `flatter` or BDD with Predicate. LLL is sufficient for RSA-512 but not for RSA-1024 with $L = 7$ bits.
- **Vector selection:** The first vector from the reduced basis is taken directly, without predicate filtering or multiple candidate verification.
- **Fault model:** Idealised — MSBs are zeroed exactly via bitwise operations, without simulating real hardware noise.

3.2 RSA-CRT Key Generation (Cell 1)

Listing 1: RSA-CRT parameter generation and correctness verification

```
nbits = 256  # RSA-512: p and q are 256 bits each
p = next_prime(2^nbits + randint(1, 10^6))
q = next_prime(2^nbits + randint(1, 10^6))
N = p * q
e = 65537
phi = (p-1)*(q-1)
d = inverse_mod(e, phi)
dp = d % (p-1);  dq = d % (q-1);  iq = inverse_mod(q, p)
```

```
def sign_rsa_crt(m, p, q, dp, dq, iq):
    sp = pow(m, dp, p)
    sq = pow(m, dq, q)
    s = sq + q * ((sp - sq) * iq % p)  # Garner's Formula, Eq.(3)
    return s, sq

m = randint(2, N-1)
s_valid, sq_real = sign_rsa_crt(m, p, q, dp, dq, iq)
assert pow(s_valid, e, N) == m  # standard RSA verification
```

Output: `RSA-CRT victim works correctly!  N has 513 bits.`

Note: N has 513 bits because p and q are generated just above 2^{256}, so $p \cdot q$ may exceed 2^{512} by 1 bit.

3.3 Classical Bellcore Attack (Cell 4)

Complete zeroing of s_q is simulated and the formula is applied:

Listing 2: Bellcore Attack – a single corrupted signature is sufficient

```
sq_faulted = 0  # total fault: sq becomes 0
s_faulted = sq_faulted + q * ((sp_b - sq_faulted) * iq % p)
recovered_p = gcd(s_correct - s_faulted, N)
# gcd(s - s_tilde, N) = p, because fault in sq and recombination mod p
    leaks factor p
```

Output obtained:

```
[SUCCESS] Bellcore attack successful! Private key compromised.
gcd(s - s_tilde, N) =
    11579208923731619542357098500868790785326998466 5...
                        (this is the value of p)
Private key d = 513832650210473883078110398484730810592141465050274 7...
```

The value returned by GCD is p, because s_q is the corrupted component. A single corrupted signature factorises the 513-bit N instantaneously.

3.4 Partial Fault Injection Simulation (Cell 2)

Fault injection is simulated by zeroing the L MSBs of s_q:

Listing 3: Generating signatures with partial fault injection on s_q

```
def generate_faulty_signatures(num_signatures, bits_to_zero):
    faulty_samples = []
    for i in range(num_signatures):
        m_i = randint(2, N-1)
        sp_i = pow(m_i, dp, p)
        sq_i = pow(m_i, dq, q)
        # Zero the L MSBs of sq (simulated fault injection)
        mask = (1 << (nbits - bits_to_zero)) - 1
```

```
        sq_tilde = sq_i & mask    # sq_tilde < 2^rho
        # CRT recombination with corrupted sq: s_tilde contains
            information about p
        s_tilde = sq_tilde + q * ((sp_i - sq_tilde) * iq % p)
        faulty_samples.append(s_tilde)
    return faulty_samples
```

Output (t=61, L=7): `Generated 61 faulty signatures with 7 MSBs zeroed.`

Note: in a real hardware attack scenario, the attacker induces the fault physically (laser, voltage glitch, or clock glitch) to force the MSBs of s_q to zero. The software simulation reproduces the exact mathematical effect without requiring specialised hardware. The model is idealised: in practice, faults are noisy and may affect other bits as well (Section 4 of the article, not implemented here).

3.5 Matrix Construction and LLL Application (Cell 3)

Following Sections 3.3–3.4.1 of the article [1], a matrix of dimension $(t+1)\times(t+1)$ is constructed:

$$M = \begin{pmatrix} 2^{\rho} & \tilde{s}_1 & \tilde{s}_2 & \cdots & \tilde{s}_t \\ 0 & -N & 0 & \cdots & 0 \\ 0 & 0 & -N & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -N \end{pmatrix} \tag{8}$$

where $\rho = \text{nbits}(q) - L$. Using $-N$ on the diagonal (instead of $-\tilde{s}_0$ from the generic SDA approach) implicitly implements the HNP reduction described in Section 3.4.1 [1]. The target vector hidden in the lattice is:

$$v = (2^{\rho}\cdot p,\ p\cdot r_1,\ p\cdot r_2,\ \ldots,\ p\cdot r_t) \tag{9}$$

where $r_i = \tilde{s}_q^{(i)} < 2^{\rho}$ are the small residues. LLL finds this vector, and p is recovered from $\gcd(v_0, N)$.

Listing 4: SDA/HNP matrix construction and LLL application

```
rho = nbits - L    # unknown bits of sq
dim = t + 1        # lattice dimension
M = Matrix(ZZ, dim, dim)
M[0, 0] = 2^rho
for i in range(1, dim):
    M[0, i] = samples[i-1]    # corrupted signatures on the first row
    M[i, i] = -N               # -N on the diagonal (HNP reduction)

M_reduced = M.LLL()            # Lenstra-Lenstra-Lov sz reduction
shortest_vector = M_reduced[0]
# p = gcd(first element of the short vector, N)
candidate_p = abs(gcd(shortest_vector[0], N))
```

Output (t=61, L=7):

```
[SUCCESS] Factor p recovered: 11579208923731619542357098500868...
Verification: N % candidate_p == 0
Attack fully successful! N has been factorised.
```

4 Results and Evaluation

4.1 Success Rate Evaluation

The PACD attack with LLL was run as a function of two parameters: the number of corrupted signatures t and the number of known bits L. Each configuration was tested over 10 independent iterations on the same RSA-512 instance.

4.1.1 Case L=32 bits — the CHES 2012 approach

Signatures (t)	Lattice size	Success Rate	Time (s)
10	11×11	100.0%	0.22
20	21×21	100.0%	0.47
30	31×31	100.0%	1.10
40	41×41	100.0%	2.63
50	51×51	100.0%	4.86

Table 1: Success rate for $L = 32$ bits on RSA-512. With 32 known bits, LLL finds the target vector with 100% success even for $t = 10$.

4.1.2 Case L=7 bits — the IACR 2024 novelty

Signatures (t)	Lattice size	Success Rate	Time (s)
30	31×31	0.0%	1.01
40	41×41	0.0%	2.92
50	51×51	40.0%	4.80
61	62×62	90.0%	8.50
70	71×71	100.0%	14.34

Table 2: Success rate for $L = 7$ bits on RSA-512. A consistent success threshold ($\geq 90\%$) is reached at $t = 61$ signatures, with $t = 70$ guaranteeing 100%.

4.2 Analysis of Results

The experimental data confirms the theoretical prediction from Section 3.4.1 of the article [1]: the target vector v has expected norm:

$$\mathbb{E}(\|v\|) \approx 0.58 \cdot \sqrt{t+1} \cdot p \cdot 2^{\rho+1} \tag{10}$$

The smaller L is ($\rho = \text{nbits}(q) - L$ larger), the more the norm of v exceeds the Gaussian heuristic $\text{gh}(\mathcal{L}(M))$, requiring a larger lattice dimension (t larger) for LLL to identify the target vector.

This explains the transition observed in Table 2: at $t = 40$ (0% success), the vector v is too long relative to the shortest lattice vector to be detected by LLL; at $t = 70$ (100% success), the lattice dimension is large enough.

4.3 Comparison with the Reference Article

Parameter	Article [1]	Current implementation
RSA dimension	1024–8192 bits	512 bits
Minimum L tested	7 bits (RSA-1024)	7 bits (RSA-512)
Signatures needed	≈2500 (RSA-1024, $L = 7$)	61–70 (RSA-512, $L = 7$)
Reduction algorithm	`flatter` + BDD	SageMath LLL
Predicate/filtering	Yes (internal GCD check)	No (first vector directly)
Fault model	Simulated (with error/noise analysis)	Simulated (idealised, no errors)
Hardware	64 cores, ≈70h	8-core macOS, <1 min
Success rate ($L = 7$)	63% with 2500 sig.	90% with 61 sig.

Table 3: Direct comparison with the experiments from the reference article.

The difference in the number of signatures needed (61 vs. 2500) does not indicate a better implementation, but a structurally easier problem: for RSA-512, $p \approx 2^{256}$, compared to $p \approx 2^{512}$ for RSA-1024. The norm of the target vector v scales linearly with p, so standard LLL is sufficient for smaller dimensions without requiring `flatter` or BDD with Predicate.

5 Countermeasures

5.1 Signature Verification — Insufficient Protection

A common misconception is that verifying $s^e \equiv m \pmod N$ before transmission provides complete protection. The article [1] demonstrates that this is false via the *safe-error* argument: if the MSBs of s_q are *naturally* zero (with probability $1/2^L$), the signature is mathematically valid and passes verification. An attacker inducing 2^L faults collects on average one undetected signature per 2^L attempts. For $L = 7$: after $2^7 = 128$ fault attempts, the attacker obtains on average one valid signature with naturally zero MSBs, sufficient for the attack.

Unlike the classical Bellcore attack, the PACD attack does not require signing the same message twice and is applicable even to schemes with randomised padding (PKCS#1 v1.5, PSS).

5.2 Multiplicative Message Blinding — Complete Protection

The effective countermeasure identified in the article [1] is **multiplicative message blinding**: the message is multiplied by $r^e \pmod N$ before signing, where r is a secret random value:

$$m' = m \cdot r^e \pmod N, \quad s' = m'^d = m^d \cdot r \pmod N \quad \Rightarrow \quad s = s' \cdot r^{-1} \pmod N \tag{11}$$

The corrupted signature becomes $\tilde{s} = \tilde{s}' \cdot r^{-1} \pmod{N}$. Without knowledge of r, the attacker cannot construct the lattice matrix, completely nullifying the effectiveness of the SDA/PACD attack.

5.3 Additive Message Blinding — Partial Protection

Additive blinding increases the effective size of $\tilde{s}_q$ by k bits (the randomisation mask). The attacker must know k additional bits beyond L, raising the required information threshold. If $k \geq 64$, the attack becomes practically infeasible with available resources [1].

6 Conclusions

This project successfully demonstrated a simplified implementation of the Barbu et al. [1] attack on an RSA-512 system in the SageMath environment.

The main conclusions are:

1. **The classical Bellcore attack** remains devastating: a single signature with $\tilde{s}_q = 0$ enables the factorisation of N via $\gcd(s - \tilde{s}, N) = p$. The process is analogous for recovering q via a fault in s_p.

2. **The PACD-to-HNP reduction** lowers the threshold from 32 bits (CHES 2012) to 7 bits (IACR 2024). This implementation implicitly achieves this reduction by constructing the matrix with $-N$ on the diagonal.

3. **The qualitative behaviour** from the article is confirmed on RSA-512: more known bits $\Rightarrow$ fewer signatures needed $\Rightarrow$ higher success rate.

4. **Signature verification** is not a sufficient protection, due to the safe-error argument: signatures with naturally zero MSBs pass verification and are collected passively by the attacker.

5. **Multiplicative message blinding** completely nullifies the PACD attack and remains the recommended countermeasure.

6. **The quantitative difference** compared to the article (61 vs. 2500 signatures for $L = 7$) reflects exclusively the difference in RSA dimension, not a superior implementation.

References

[1] Guillaume Barbu, Laurent Grémy, and Roch Lescuyer. *Revisiting PACD-based Attacks on RSA-CRT*. IACR Cryptology ePrint Archive, Report 2024/1125, 2024. https://eprint.iacr.org/2024/1125

[2] Dan Boneh, Richard A. DeMillo, and Richard J. Lipton. *On the Importance of Checking Cryptographic Protocols for Faults*. In *Advances in Cryptology – EUROCRYPT '97*, LNCS vol. 1233, pp. 37–51. Springer, 1997. https://link.springer.com/chapter/10.1007/3-540-69053-0_4

[3] Pierre-Alain Fouque, Nicolas Guillermin, Delphine Leresteux, Mehdi Tibouchi, and Jean-Christophe Zapalowicz. *Attacking RSA-CRT Signatures with Faults on Montgomery Multiplication*. In *CHES 2012*, LNCS vol. 7428, pp. 447–462. Springer, 2012. https://link.springer.com/chapter/10.1007/978-3-642-33027-8_26

[4] Arjen K. Lenstra, Hendrik W. Lenstra Jr., and László Lovász. *Factoring Polynomials with Rational Coefficients*. *Mathematische Annalen*, 261:515–534, 1982. https://doi.org/10.1007/BF01457454

[5] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*. *Communications of the ACM*, 21(2):120–126, 1978. https://doi.org/10.1145/359340.359342